

modelle/ — der Verfahrensvergleich und seine Deutung

Fünf Dateien beantworten die Forschungsfrage: Welches Verfahren erzielt die höchste Prognosegüte, welche Merkmale tragen sie, und was kostet sie.

BACHELORARBEIT

SFFD-Einsatzprognose

PHASE

CRISP-DM 4 und 5

ERGEBNIS

Verfahrensvergleich · SHAP · 10 Abbildungen

Was dieser Ordner leistet

STRANG 1**m02_menge.py**

Wie viele Einsätze? Ridge, RF, XGBoost gegen das Poisson-GLM

**STRANG 2****m03_struktur.py**

Welche Art dominiert? RF und XGBoost gegen das Logit

**DEUTUNG****m04_shap.py**

Welche Merkmale tragen — Attribution und Ablation

**DARSTELLUNG****m05_abbildungen.py**

Zehn Abbildungen, ausschließlich aus CSV gelesen

DIE VIER UNTERFRAGEN DER ARBEIT

- **UF1** — Welche Faktorgruppe trägt wie viel? (m04)
- **UF2** — Welches Verfahren erzielt die höchste Güte? (m02, m03)
- **UF3** — Was kostet Training und Inferenz? (m02, m03 → A4, A9)
- **UF4** — Was folgt daraus für die Modellauswahl? (m04, v3 → A3)

DIE REGEL, DIE DEN ORDNER ZUSAMMENHÄLT

Ein Lauf, **eine** Spezifikation. Nichts wird aus einem früheren Lauf wiederverwendet, kein Zwischenstand eingelesen, kein Schalter ändert das Modell. Wer die Spezifikation ändert, rechnet neu — sonst wären still Parameter aus einer anderen Welt im Ergebnis.

Die fünf Dateien

DATEI	ROLLE	LÄUFE	ZEILEN
config_modelle.py	Suchräume, Budget, Startwert	—	110
m02_menge.py	Mengenstrang: 2 Zielgrößen × 3 Verfahren	300	840
m03_struktur.py	Strukturstrang: 1 Zielgröße × 2 Verfahren	100	732
m04_shap.py	Interpretation: Beiträge und Ablation	—	690
m05_abbildungen.py	Zehn Abbildungen für Kapitel 7	—	1 103

REIHENFOLGE

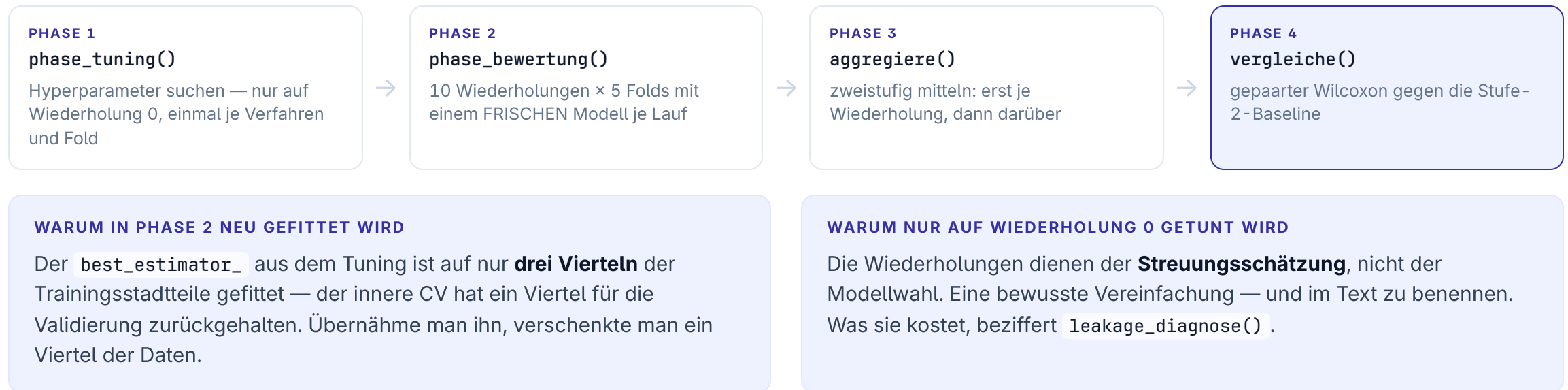
m02 und m03 sind unabhängig voneinander. m04 setzt beide voraus, m05 setzt m02, m03, m04 sowie v1 und v3 voraus.

ZWEI BETRIEBSARTEN

m02 und m03 kennen das Argument `holdout` . Ohne es werden die sechs Hold-out-Stadtteile **zu Beginn und unwiderruflich** herausgefiltert — alles Folgende kann sie nicht mehr sehen, auch nicht versehentlich.

Das Vier-Phasen-Muster

m02 und m03 sind identisch aufgebaut. Wer eine der beiden Dateien verstanden hat, kennt die andere.



Drei Fallstricke, die in beiden Strängen gelten

Sie sind der Grund, warum die Zahlen glaubwürdig sind — und die wahrscheinlichsten Prüfungsfragen.

1 · DER INNERE CV

RandomizedSearchCV nimmt voreingestellt KFold und schneidet **nach Zeilen**.

- Ein Stadtteil hat 132 Zeilen; die Strukturmerkmale sind innerhalb eines Jahres konstant.
- Ohne Gruppierung stünden faktisch dieselben Zeilen in innerem Training und innerer Validierung.
- **Man sieht es den Zahlen nicht an — sie wären nur zu gut.**

2 · ZWEISTUFIG MITTELN

Die 50 Fold-Ergebnisse sind **nicht unabhängig**.

- Es sind dieselben 29 Stadtteile in zehn Gruppierungen.
- Ein Intervall aus `std_folds/v50` wäre zu eng.
- Maßgeblich ist `std_wiederholungen` — die Streuung der zehn Wiederholungsmittel.
- Beide Spalten wandern mit, damit der Unterschied sichtbar bleibt.

3 · DIE HOLD-OUT-SPERRE

Sechs Stadtteile bleiben bis zur Schlussbewertung unberührt.

- Der Filter steht in `main()`, **bevor** irgendetwas rechnet.
- Er ist konstruktiv, nicht disziplinarisch: Was gefiltert ist, kann kein späterer Codeteil versehentlich lesen.
- Die Schlussbewertung ist **eine** Messung an **sechs** Einheiten — kein Mittelwert, keine Streuung.

DATEI

config_modelle.py

Suchräume, Budget und Startwert. Berührt keine einzige Datei auf der Platte.

110

Zeilen

4

Konstanten

100

Tuning-Budget

Die Trennlinie zu prep/config.py

Nicht „Daten gegen Modelle“ — sondern: was geschrieben wird gegen was gerechnet wird.

PREP/CONFIG.PY HÄLT FEST ...

- welche Spalten die Panels haben
- wie `fold` und `ist_holdout` belegt sind
- welche Prädiktoren und Klassen es gibt
- — also alles, was **in die Datei geschrieben** wird

CONFIG_MODELLE.PY HÄLT FEST ...

- die Suchräume je Verfahren
- `TUNING_BUDGET` und `RANDOM_STATE`
- `WIEDERHOLUNGEN = 10`
- — also alles, was nur **beim Rechnen** gilt

WARUM DIE PRÄDIKTOREN NICHT HIERHER WANDERN

Die Modellskripte **lesen** diese Festlegungen, sie treffen sie nicht. Zöge man sie hierher, gäbe es zwei Dateien, die zwingend übereinstimmen müssen — genau die Fehlerquelle, die der Aufbau vermeiden soll.

DIE BEKANNTE SCHWÄCHE

Vier Dateien der Vorprüfung holen `WIEDERHOLUNGEN` bzw. `RANDOM_STATE` von hier — der Pfeil zeigt **rückwärts**. Fachlich richtig ist, dass es nur *eine* Definition gibt; strukturell säße sie besser in `prep/config.py`.

Budget und Suchräume

Zwei Zahlen, die beide hergeleitet und nicht gewählt sind.

WARUM BUDGET 100

Bergstra & Bengio (2012) geben die geschlossene Form

$$P = 1 - (1 - v/V)^T$$

- für $v/V = 0,05$: $T = 50 \rightarrow 92,3 \%$, $T = 100 \rightarrow 99,4 \%$
- Die Formel enthält die **Dimension nicht** — deshalb bekommen Ridge (1 Parameter) und XGBoost (7 Parameter) dasselbe Budget.
- Die verbreitete Angabe „60 Ziehungen“ steht nicht in dem Papier.

WARUM VIER SUCHRÄUME ERWEITERT WURDEN

- Erweitert wurde, wo der beste Wert **an der Grenze** lag — bei sechs von sieben geprüften Parametern.
- Das ist eine Aussage über die **Suche**, nicht über das Ergebnis: Lag das Optimum an der Grenze, war die Grenze falsch gesetzt.
- **Nicht** erweitert: `max_features`, `min_samples_leaf`, `subsample`, `colsample_bytree` — ihre Grenzen sind natürlich, dahinter existiert nichts.

DER WICHTIGSTE FUND

`max_depth` lief bei XGBoost von 3 bis 10. Im Strukturstrang wählten **vier von fünf Folds die Untergrenze 3**. Mit geöffneter Grenze wählen sie 2, 2 und 1 — das Modell wollte flacher sein, als es durfte. Zusammen mit `reg_lambda` ist das die plausibelste Erklärung dafür, dass die Baumverfahren in der Kreuzvalidierung gewinnen und auf dem Hold-out verlieren.

DATEI

m02_menge.py

Der Mengenstrang. Zwei Zielgrößen × drei Verfahren × 10 Wiederholungen × 5 Folds.

840

Zeilen

16

Funktionen

300

Läufe

verfahren(name, n_jobs) und suchraum(name)

Wie die drei Pipelines gebaut werden — und warum sie unterschiedlich aussehen dürfen.

VERFAHREN(NAME, N_JOBS)

EIN Verfahrensname, optional n_jobs

AUS Pipeline ohne Hyperparameter

- **Ridge** bekommt StandardScaler und log-Zieltransformation *in* der Pipeline — der L2 - Strafterm behandelt alle Koeffizienten gleich.
- Die Baumverfahren bekommen **keine** Zieltransformation: sie sind skalenunempfindlich, und eine transformierte Zielgröße machte die Gütemaße unvergleichbar.

SUCHRAUM(NAME)

EIN Verfahrensname

AUS dict Parametername → scipy-Verteilung

- Die Config hält die Räume als Tupel ("loguniform", a, b) , damit sie ohne scipy lesbar bleiben.
- Der **Präfix** hängt am Pipeline-Aufbau: bei Ridge liegt der Schätzer zwei Ebenen tief, bei den Baumverfahren direkt.

DIE VERLUSTFUNKTION — EINE KORREKTUR, KEIN FEINTUNING

Bis 06.08.2026 rechneten beide Baumverfahren mit dem **quadratischen Fehler auf rohen Zähldaten**, während Baseline und Ridge multiplikativ arbeiteten. Bei Einsatzzahlen von 6 bis 280 gewichtet der quadratische Fehler einen Fehler von 20 in Tenderloin genauso wie in Seacliff, wo er das Dreifache des Gesamtwerts ausmacht. Seither: `reg:tweedie` für XGBoost, `criterion="poisson"` für den Random Forest. Dass scikit-learn kein Tweedie für Wälder kennt, ist selbst ein berichtbarer Befund über das Verfahren.

tune(name, train, ziel)

Der Fallstrick, der die ganze Arbeit entwerfen kann.

EIN Trainingsrahmen des Folds, Verfahren, Zielgröße

AUS die PARAMETER als dict — nicht das Modell

DER INNERE CV MUSS GRUPPIEREN

- `RandomizedSearchCV` nimmt voreingestellt `KFold`.
- Ein Stadtteil hätte dann etwa 100 Zeilen im inneren Training und 32 in der inneren Validierung.
- Da die Strukturmerkmale innerhalb eines Jahres konstant sind, wären das faktisch **dieselben Zeilen**.
- Die Hyperparameter würden auf einen geleakten Schätzwert optimiert — der Vorteil des äußeren Stadtteil-Splits wäre verspielt.

WARUM DIE PARAMETER ZURÜCKKOMMEN

Nicht `best_estimator_`: der ist auf dem inneren Trainingsanteil gefittet und verschenke ein Viertel der Daten.

DER ZWEITE FALLSTRICK

Der Schätzer läuft **einkernig**, parallelisiert wird allein die Suche. Zuvor stand `n_jobs=-1` an beiden Stellen — die Prozesse blockierten sich gegenseitig; ein Probelauf stand nach 15 Minuten noch in Phase 1.

ein_lauf(...)

Ein Fit, eine Vorhersage — und vier Messgrößen, die daneben mitlaufen.

EIN Trainings- und Testrahmen, Verfahren, Parameter, Zielgröße, auch_parallel

AUS dict mit Gütemaßen, Laufzeiten, n_negativ, y_hat_min, Extrapolationsanteil

DIE ZEITMESSUNG

- Gemessen wird **um fit und predict herum**, nicht um die ganze Funktion — sonst steckte die Metrikberechnung mit in der Zahl.
- **Einkernig** für alle drei Verfahren, damit Unterfrage 3 vergleichbar bleibt.
- `auch_parallel=True` misst denselben Fit zusätzlich über alle Kerne — die Differenz ist der Parallelisierungsgewinn.

DIE MITLAUFENDEN DIAGNOSEN

- `n_negativ` und `y_hat_min`: Ridge auf $\log(1+y)$ kann nach `expm1` Werte unter null liefern. **Nicht gekappt** — das wäre ein Eingriff —, aber ausgewiesen.
- `parallel_abweichung_max`: Ändert die Kernzahl das *Ergebnis*? Bei XGBoost ja, erheblich; bei Ridge und RF null.
- `ueberanpassung_*`: dieselbe Güte auf den **Trainings**stadtteilen.

WIE DIE ÜBERANPASSUNGSZAHL NICHT ZU LESEN IST

Ein Random Forest mit `min_samples_leaf = 1` interpoliert seine Trainingsdaten **konstruktionsbedingt** — ein Trainings- R^2 von 0,98 ist dort erwartbar und kein Beweis krankhafter Überanpassung. Der Abstand ist zwischen *Konfigurationen desselben Verfahrens* aussagekräftig, nicht als Verhältnis zwischen Verfahren.

Die vier Phasenfunktionen

PHASE_TUNING(PANEL, SELTEN)

EIN Panel der Entwicklungsstadtteile

AUS tuning.csv mit 30 Zeilen

- **Keine Wiederverwendung:** tuning.csv ist ein Ergebnis dieses Laufs, kein Eingang.
- Gesucht wird über (Verfahren × Fold) = **15** Durchgänge; beide Zielgrößen erhalten denselben Satz.
- `tuning_sekunden` steht deshalb bei beiden Zielgrößen gleich — eine Summe über alle 30 Zeilen zählt doppelt.

PHASE_BEWERTUNG(PANEL, PARAMETER, SELTEN)

EIN Panel, Parametertabelle aus Phase 1

AUS menge_folds.csv mit 300 Zeilen

- 10 Wiederholungen × 5 Folds × 3 Verfahren × 2 Zielgrößen.
- Trainiert wird je Fold auf **allen** Trainingsstadtteilen — mit einem frischen Modell.

AGGREGIERE(FOLDS)

- Zweistufig mitteln: erst je Wiederholung über die 5 Folds, dann die Streuung dieser zehn Werte.
- Liefert `std_folds` **und** `std_wiederholungen`.

DIE BEIDEN STATISTIKHELPER

- `_gepaart(a, b)` — Wilcoxon plus mittlere Differenz, Konfidenzintervall und gewonnene Läufe. Bei RMSE ist klein besser, `b - a` ist der Vorteil von `a`.
- `_holm(p)` — Holm-Bonferroni: p-Werte aufsteigend, kleinster gegen α/m . Uniform stärker als Bonferroni bei gleicher Fehlerkontrolle.

vergleiche(folds, baselines)

Phase 4 — hier entsteht die Primäraussage der Arbeit.

EIN menge_folds.csv und die Baseline-Läufe aus v1_baselines.py

AUS vergleich.csv

ZWEI ROLLEN

- **primär** — jedes Verfahren gegen die Stufe-2-Baseline: $3 \times 2 = 6$ Tests. **Keine Familie**, weil jede Frage vorab einzeln formuliert ist; keine Korrektur.
- **sekundär** — jedes Verfahrenspaar: $3 \text{ Paare} \times 2 \text{ Zielgrößen} = 6$ Tests. **Eine Familie**, darauf Holm-Bonferroni.

ZWEI TESTSTUFEN

- **wiederholung**, $n = 10$ — **der Primärtest**. Die 50 Einzelläufe sind Pseudoreplikation und liefern zu kleine p-Werte.
- **lauf**, $n = 50$ — ausdrücklich als **Sensitivität** geführt, nicht als Ergebnis.

WARUM IMMER DREI ZAHLEN DANE BEN STEHEN

Auch die zehn Wiederholungsmittel sind nicht unabhängig — es bleiben 29 Einheiten. Das berichtete Konfidenzintervall ist daher **enger als die wahre Unsicherheit** (Nadeau & Bengio 2003). Deshalb stehen mittlere Differenz, Intervall und gewonnene Läufe **immer** neben dem p-Wert — unabhängig davon, wie er ausfällt. Der Sinn: Ein Ergebnis darf nicht an einer einzigen Schwelle hängen.

hold_out() und leakage_diagnose()

Die beiden Funktionen, die das eigene Vorgehen prüfen.

HOLD_OUT(PANEL, PARAMETER, FOLDS, SELTEN)

EIN vollständiges Panel, Parametertabelle aus Phase 1

AUS holdout.csv mit Spalte fold_der_parameter

- Auf 29 Stadtteilen trainieren, auf 6 bewerten — **einmalig**.
- Welche Parameter? Das Tuning liefert fünf Sätze. Gewählt ist der Satz des Folds mit dem niedrigsten RMSE in Wiederholung 0 — deterministisch und ausschließlich aus Entwicklungsdaten.
- **Beide Baselines laufen mit:** Ein RMSE von 23,7 ist ohne Bezugspunkt keine Aussage.

LEAKAGE_DIAGNOSE(FOLDS, BASELINES)

EIN menge_folds.csv und die Baseline-Läufe

AUS Vorsprung in W0 gegen W1–9

- Getunt wird einmal, auf Wiederholung 0. Dort ist der Vorsprung sauber gemessen.
- In W1–9 waren im Mittel **78 %** der Teststadtteile in der Suchmenge. Wäre der Effekt bedeutsam, müsste der Vorsprung dort systematisch größer ausfallen.
- Bewusst als **schwache** Diagnose zu lesen: W0 ist auch eine andere Aufteilung, der Unterschied ist konfundiert.

WIE DAS HOLD-OUT ZU BERICHTEN IST

Als **eine** Messung an **sechs** Einheiten — kein Mittelwert, keine Streuung. Die Zahl ist deutlich unsicherer als die Kreuzvalidierungswerte und darf nicht als deren Bestätigung gelesen werden.

Die Hilfsfunktionen und main(argv)

`_REIN_PYTHON(P)`

- Wandelt NumPy-Skalare in native Typen, **bevor** sie nach JSON gehen.
- `np.float64` erbt von `float` und überlebt `json.dumps` zufällig — `np.int64` erbt **nicht** von `int`.
- Mit `default=str` würde aus 287 die Zeichenkette „287“, und `set_params` bräche ab — mitten im mehrstündigen Lauf, nach dem Tuning.

`_PARAMETER_JE_FOLD(PARAMETER) · PAAR(...)`

- Zwei reine Umformer: Nachschlagetabelle aus `tuning.csv`, und eine Vergleichszeile für `vergleich.csv`.

`EXTRAPOLATIONSANTEIL(TRAIN, TEST)`

- Anteil der Testzeilen außerhalb des Trainings-Wertebereichs.
- Erfasst nur die **Spanne je Merkmal**, nicht unbekannte Kombinationen — das echte Extrapolationsproblem ist also eher größer.
- Erklärt später, warum ein Fold aus der Reihe fällt.

`MAIN(ARGV)`

- Führt die vier Phasen und schreibt fünf CSV-Dateien.
- Ohne das Argument `holdout` werden die Hold-out-Zeilen **zu Beginn** herausgefiltert.

DATEI

m03_struktur.py

Der Strukturstrang. Spiegelt m02 — dieselben Funktionen, dieselbe Reihenfolge. Hier zählen nur die Unterschiede.

732

Zeilen

20

Funktionen

100

Läufe

Was anders ist als in m02

ASPEKT	M02_MENGE.PY	M03_STRUKTUR.PY
Zielgröße	anzahl_einsaetze, einsaetze_je_1000_ew	dominante_einsatzart — vier ungeordnete Klassen
Verfahren	Ridge, Random Forest, XGBoost	nur Random Forest und XGBoost
Stufe-2-Baseline	Poisson-GLM mit Offset	multinomiales Logit
Gütemaß	RMSE (getestet), MAE und R ² nachrichtlich	Macro-F1 (getestet), Macro-AUROC, Accuracy nachrichtlich
Tuning-Scoring	RMSE	f1_macro
Mehrfachvergleich	Holm über 6 sekundäre Tests	kein Holm — nur ein sekundärer Test
Läufe	300	100

WARUM RIDGE FEHLT

Eine lineare Regression hat auf einer **nominalen** Zielgröße keine Entsprechung. Das Gegenstück wäre das multinomiale Logit — und das ist bereits die Baseline.

WARUM KEIN HOLM

Die sekundäre Familie besteht aus einem **einzigem** Test (RF gegen XGBoost); eine Korrektur über einen Test ist die Identität. Regression und Klassifikation sind getrennte Testfamilien — das ist in Kapitel 7 zu benennen, weil dieser Vergleich unkorrigiert gegen $\alpha = 0,05$ läuft.

Drei Fallstricke, die es in m02 nicht gibt

1 · KLASSENGEWICHTE STATT RESAMPLING

Die Klassenverteilung ist stark schief — **79 % Fehllalarm**.

- Random Forest: `class_weight="balanced"` als Hyperparameter.
- XGBoost: `sample_weight` beim Fit — das Verfahren kennt keinen solchen Parameter. Dafür `_gewichte()`.
- **Kein SMOTE**, kein Over- oder Undersampling: Das wäre ein Eingriff in die Datenverteilung und bräche die Vergleichbarkeit mit den Baselines.

2 · LABEL-ENCODER EINMAL GLOBAL

`XGBClassifier` erwartet Integer-Labels 0...3 — `kodierte()` liefert sie.

- Das Mapping folgt der **globalen** Reihenfolge `KLASSEN`, nicht den gerade vorliegenden Daten.
- Ein je Fold gefitteter `LabelEncoder` verschöbe in einem Fold ohne Brand die Zahlen.
- Die Wahrscheinlichkeitsspalten zeigten danach auf die **falschen Klassen** — ohne Fehlermeldung.

3 · MACRO-AUROC KANN UNDEFINIERT SEIN

Fehlt eine Klasse im Testfold, ist sie nicht berechenbar.

- Dann als **fehlend** führen — **nicht** durch 0,5 oder 0 ersetzen.
- Ein erfundener Wert zöge den Mittelwert nach unten und sähe wie eine Messung aus.
- `labels=klassen_model1` bringt die Spalten in die Reihenfolge, die das Modell benutzt hat — der zweite Teil von Fallstrick 2.
- `zero_division=0` bei Macro-F1 muss gesetzt bleiben, sonst bricht der Lauf ab.

Die eigenen Funktionen im Detail

TUNE(NAME, TRAIN)

EIN Trainingsrahmen des Folds, Verfahren

AUS die Parameter als dict

- Der Fallstrick aus m02 gilt unverändert: Der innere CV **muss** nach Stadtteil gruppieren.
- **f1_macro statt Accuracy**, weil die Mehrheitsklasse allein über 0,8 Accuracy erreicht. Ein darauf optimiertes Tuning wählte Modelle, die die drei seltenen Klassen ignorieren — genau das, was die Fragestellung nicht will.

SUCHRAUM(NAME)

EIN Verfahrensname

AUS dict Parametername → Verteilung

- Beide Verfahren sind hier **nackte Schätzer** statt Pipelines — keine Skalierung nötig.
- Eine Ausnahme: `tweedie_variance_power` steuert die Verlustfunktion der *Regression* und ist bei `multi:softprob` bedeutungslos. XGBoost nähme ihn stillschweigend an — ein Sechstel des Budgets auf einer wirkungslosen Dimension.

EIN_LAUF(...) UND FITTE(...)

- Die Wahrscheinlichkeiten für die AUROC kommen aus einem **zweiten** Aufruf, damit `inferenz_sekunden` die reine Klassenvorhersage misst und vergleichbar bleibt.
- `fitte()` kapselt, dass XGBoost die Gewichte beim Fit bekommt und der RF sie als Hyperparameter hat.

HOLD_OUT(PANEL, PARAMETER, FOLDS)

- Gewählt wird der Parametersatz des Folds mit dem **höchsten** Macro-F1 in Wiederholung 0.
- Das Baseline-Modell stammt seit 10.08.2026 aus `v1_baselines.logit_glm()` — vorher war es hier ein zweites Mal nachgebaut.

DATEI

m04_shap.py

Interpretation. Welche Merkmale tragen die Vorhersage — und was wären sie wert, wenn man sie weglässt?

690

Zeilen

10

Funktionen

9

Ergebnisdateien

Attribution und Ablation

Zwei Antworten auf Unterfrage 1 — und der Grund, warum es beide braucht.

ATTRIBUTION

Welcher Anteil der SHAP- bzw. Koeffizientenmasse entfällt auf eine Faktorgruppe?

- Sagt, wie ein Modell seine **Aufmerksamkeit verteilt**.
- Summiert sich je Modell auf 100 %.
- Dateien: `gruppen.csv` , `faktorgruppen_menge.csv`

ABLATION

Was kostet es, die Gruppe wegzulassen?

- Sagt, was die Gruppe **wert** ist.
- Jede Gruppe wird einmal entfernt, alles andere bleibt gleich. Die Verschlechterung ist der Beitrag.
- Datei: `ablation_faktorgruppen_mittel.csv`

WARUM DIE ZWEITE FRAGE DIE HÄRTERE IST

Ein Merkmal kann **viel Masse binden und trotzdem ersetzbar sein**, weil ein anderes dieselbe Information trägt. Unter Kollinearität laufen Attribution und Ablation deshalb auseinander — und nur die Ablation trennt „bekommt Aufmerksamkeit“ von „ist unverzichtbar“.

Die eine Regel

SHAP wird nur für Modelle gerechnet, die ihre Stufe-2-Baseline schlagen.

SCHLAGEN_DIE_LATTE(VERGLEICH)

EIN vergleich.csv beider Stränge

AUS Menge der zugelassenen Kombinationen

- Grundlage ist der **Primärtest** auf den Wiederholungsmitteln.
- Verlangt werden **beide** Bedingungen: mittlere Differenz zugunsten des Verfahrens **und** signifikanter Test.
- Ein positiver Mittelwert allein wäre zu wenig.

RUHIGSTER_FOLD(FOLDS)

- Gerechnet wird auf **einem** Fold — dem mit dem geringsten Extrapolationsanteil in Wiederholung 0.
- Dort liegen die wenigsten Testzeilen außerhalb des gelernten Wertebereichs; die Beiträge beruhen also am ehesten auf **Interpolation**.

WARUM DIE REGEL NÖTIG IST

Für ein Modell, das seine Baseline nicht schlägt, **erklärt man Rauschen**. Eine Abbildung, die Beiträge zeigt, wo kein Signal ist, ist schlimmer als keine Abbildung.

WARUM DAS NICHT NACH ROSINENPICKEN AUSSIEHT

Die übersprungenen Modelle stehen **mit Begründung** in `uebersprungen.csv`. Die Auswahl ist damit nachvollziehbar statt selektiv.

DIE FOLGE IM MENGENSTRANG

Dort bleibt der Attributionsblock **leer** — kein Vergleichsverfahren schlägt das Poisson-GLM. Das ist ein Ergebnis, kein Fehler. Die Antwort auf UF1 kommt deshalb aus `faktorgruppen_baseline()`.

`_beitraege(modell, X, name)`

Wie die Beiträge je Merkmal entstehen — und warum XGBoost einen eigenen Weg geht.

EIN gefittetes Modell, Merkmalsmatrix, Verfahrensname

AUS Reihe Merkmal → mittlerer absoluter Beitrag

DREI WEGE, EIN ERGEBNIS

- **Ridge und GLM:** standardisierte Koeffizienten — der direkte Gegenwert zu SHAP-Beiträgen, kein Explainer nötig.
- **Random Forest:** `TreeExplainer`, exakt statt approximiert.
- **XGBoost:** `pred_contribs=True` — XGBoost bringt TreeSHAP selbst mit.
- Mehrklassige Ausgaben werden über die Klassen gemittelt: Die Frage lautet „welche Faktorgruppe trägt“, nicht „für welche Klasse“.

WARUM DER UMWEG BEI XGBOOST

`shap.TreeExplainer` kann den mehrklassigen `base_score` von XGBoost 3.x nicht lesen und bricht mit `could not convert string to float` ab. `pred_contribs=True` liefert **exakt dieselben Werte**, gerechnet vom selben Algorithmus — kein Näherungsverfahren, nur ein anderer Aufrufweg.

DER FALLSTRICK BEIM DEUTEN

Die Strukturmerkmale sind untereinander korreliert. SHAP verteilt den Beitrag dann auf mehrere Merkmale, und einzelne Werte sind nicht sinnvoll deutbar — „`median_haushaltseinkommen` trägt 8 %“ wäre **Scheinpräzision**. Deshalb wird auf die drei Faktorgruppen des Exposés verdichtet; Größenkontrolle und Saison werden getrennt ausgewiesen.

Die beiden Ablationen

ABLATION_EXPOSITION(...)

EIN Panel, tuning.csv des Mengenstrangs

AUS ablation_exposition.csv

- Entfernt **einen** Baustein: Die Baumverfahren passen direkt auf `anzahl_einsaetze` an statt über die Rate.
- Gemessen: ohne Expositionsbehandlung **RF 67,7** und **XGBoost 61,7** RMSE — mit ihr **36,4** und **35,7**.
- Der Spezifikationsunterschied ist damit ein **Vielfaches** des Verfahrensunterschieds.

ABLATION_FAKTORGRUPPEN(...)

EIN beide Panels, tuning.csv beider Stränge

AUS ablation_faktorgruppen.csv

- Jede Faktorgruppe wird einmal weggelassen, alles andere bleibt gleich.
- **Mengenstrang:** abladiert wird das Poisson-GLM — es ist das beste Modell des Strangs und hat keinen Hyperparameter. Dann ändert sich ausschließlich die Merkmalsmenge.
- **Strukturstrang:** RF und XGBoost, die beide die Baseline schlagen.

DIE EINSCHRÄNKUNG, DIE BERICHTET WERDEN MUSS

Bei den Baumverfahren stammen die Hyperparameter aus dem **vollen** Merkmalssatz und werden nicht neu gesucht — sonst änderte sich zweierlei gleichzeitig. Die gemessene Verschlechterung enthält dadurch einen Anteil, der auf eine nicht mehr passende Einstellung entfällt.

_ABLATION_AUSWERTEN(ROH)

Dreht das **Vorzeichen** so, dass ein positiver Wert immer „Verschlechterung durch Weglassen“ heißt — bei RMSE ist klein besser, bei Macro-F1 groß. Ohne diese Drehung liest man eine der beiden Tabellen genau falsch herum.

Die drei übrigen Auswertungen

FAKTORGRUPPEN_BASELINE(...)

- Beantwortet UF1 für den Mengenstrang — **aus der Baseline**, weil dort jedes Vergleichsverfahren übersprungen wird.
- Das ist kein Notbehelf: Das beste Modell des Strangs **ist** das Poisson-GLM.
- Vergleichbar gemacht über $|\text{Koeffizient}| \times \text{Standardabweichung}$ — sonst hinge die Größe an der Einheit (Einkommen in Dollar bekäme einen winzigen Koeffizienten).

EXTRAPOLATION_AUFSCHLUESSELN(...)

- Macht aus einer Plausibilitätsaussage eine Zahl: je Merkmal, je Stadtteil, und der Spearman-Zusammenhang zwischen Extrapolationsanteil und Fehler.

_VIF(PANEL)

- Zwei Bezugsmengen, und beide werden ausgewiesen.
- Absicht war, jede Merkmalskombination **einmal** zu zählen. Ein `drop_duplicates()` leistet das nicht: Seit der Kriminalitätsindex monatlich rolliert, sind 3 757 von 3 828 Zeilen eindeutig.
- `stadtteil_jahr` ist die Ebene, auf der die Merkmale tatsächlich variieren — **diese** Zahl gehört in den Text.

DIE ABGRENZUNG, AUF DIE ES ANKOMMT

Verboten wäre, die **Testmenge** nach Extrapolationsgrad zu schneiden und darin nach Verfahrensunterschieden zu suchen — das wäre ein nachträglicher Zuschnitt der Auswertung. Hier bleibt die Einheit der **Fold**, und die Frage lautet, warum Folds unterschiedlich schwer sind. Die Primäraussage bleibt unberührt.

DATEI

m05_abbildungen.py

Zehn Abbildungen für Kapitel 7. Dieses Skript rechnet nichts — es liest nur.

1 103

Zeilen

21

Funktionen

10

Abbildungen

Warum gepaart dargestellt wird

Der Neuschnitt vom 07.08.2026 — und der messbare Grund dafür.

WAS DER ERSTE SATZ ZEIGTE

- Boxplots der **Rohwerte** je Verfahren.
- Die 50 Läufe unterscheiden sich darin, *welche* Stadtteile im Testfold liegen.
- Bayview hat ein Vielfaches der Einsätze von Seacliff — der RMSE schwankt zwischen **13 und 76**, unabhängig vom Verfahren.
- Streuung der Rohwerte: 12,4 bis 15,5. Verfahrensunterschied: **rund 2**.

WAS DER NEUSCHNITT ZEIGT

- Jedes Verfahren sieht **dieselben** Folds.
- Bildet man die Differenz je Lauf, kürzt sich die Fold - Streuung heraus.
- Streuung der gepaarten Differenz über die zehn Wiederholungsmittel: **2,4 bis 4,3**.
- Es ist **dieselbe Paarung**, auf der auch der Wilcoxon - Test beruht — die Abbildung zeigt die getestete Größe.

DER ZWEITE, EINFACHERE FEHLER DES ALTEN SATZES

Balken ab null, während sich alles zwischen 33,98 und 36,51 abspielte. Die Unterschiede lagen in den obersten sechs Prozent der Bildhöhe.

Die zehn Abbildungen

	ABBILDUNG	ZEIGT	BEANTWORTET
A1	gegen_baseline	gepaarte Differenz zur Stufe - 2 - Baseline, ein Punkt je Wiederholung	Primäraussage
A2	foldstruktur	Rohwerte je Fold — die Streuung stammt aus dem Fold	Begründung für A1
A3	spezifikation	Verfahrenswahl gegen Spezifikationswahl	UF4
A4	laufzeit_guete	einkernige Trainingszeit gegen Güte	UF3
A5	holdout	die einmalige Schlussbewertung	Validierung
A6	faktorgruppen	Anteil je Faktorgruppe, gestapelt auf 100 %	UF1
A7	extrapolation	Extrapolationsanteil gegen Fehler, mit Spearman - ρ	Limitation
A8	hyperparameter	Lage der fünf Fold - Parametersätze im Suchraum	Kapitel 8
A9	parallelisierung	Faktor, um den das parallele Fitten schneller wird	UF3, zweite Hälfte
A10	qq_residuen	QQ - Diagramm der Residuen	Auflage 10.08.

Anforderungen an die Darstellung

Gestaltung war im Gutachten ein eigenes Bewertungskriterium.

SATZ UND FORMAT

- **PDF, nicht PNG** — Rasterbilder werden im Druck unscharf.
- In der **Endgröße** erzeugen, nicht in LaTeX schrumpfen: sonst steht dort 5-pt-Schrift. Mindestens 9 pt.
- **Keine Titel** in der Abbildung — die Bildunterschrift in LaTeX ist der Titel.
- Verfahren zusätzlich über **Schraffur und Marker** unterscheiden, nicht allein über Farbe.

INHALTLICHE PFLICHTEN

- Achsen mit Einheit und deutschem **Dezimalkomma**.
- Bei Differenzen und R^2 die **Nulllinie** einzeichnen — das Vorzeichen ist die Aussage.
- An jeder Differenzachse muss stehen, **welche Seite besser ist**: bei RMSE links, bei Macro-F1 rechts.
- Bei jeder Streuung benennen, **worüber** sie gebildet ist — zehn Wiederholungsmittel, nicht 50 Einzelläufe.

DIE PRÜFFRAGE, DIE AM EHESTEN GESTELLT WIRD

„**Schneidet in A1 die Nulllinie eine der Boxen?**“ — Dann darf im Text **kein** Unterschied zur Baseline behauptet werden, den der Test nicht deckt. Die Abbildung und der Text müssen dieselbe Aussage tragen.

Die Hilfsfunktionen

Formatierung, die inhaltlich trägt.

`_KOMMA(...)` · `_PROZENT(...)` · `_SEKUNDEN(WERT)`

- `stellen` ist **nicht kosmetisch**: Macro-F1 liegt zwischen 0,328 und 0,334 — mit zwei Nachkommastellen stünde an allen Achsenmarken „0,33“.
- `vorzeichen` setzt auf Differenzachsen ein explizites Plus; sonst liest sich „2,5“ wie ein Absolutwert statt wie ein Abstand.
- `_sekunden` : zwei Nachkommastellen reichen für die Ensembles (5,83 s), nicht für Ridge (0,011 s).

`_LAGE_IM_SUCHRAUM(NAME, PARAMETER, WERT)`

- Normiert einen gefundenen Wert auf 0...1 in **seinem** Suchraum — `alpha` läuft über sechs Zehnerpotenzen, `subsample` über 0,4 Einheiten.
- Fällt ein Wert aus seinem Raum, gibt es `None` : dann hat sich der Suchraum seit dem Lauf geändert, und die Zeile **fehlt**, statt eine falsche Lage vorzutäuschen.

`_GEPAARTE_DIFFERENZ()`

- Liefert je Verfahren die zehn Wiederholungsmittel der Differenz zur Baseline.
- Gepaart wird auf (`wiederholung`, `fold`) — also auf **identischen Testzeilen**.
- Damit zeigt die Abbildung die *getestete* Größe und nicht eine andere, die zufällig ähnlich aussieht.

`_HYPERPARAMETER_LAGEN()` · `_FAKTORGRUPPEN_BALKEN()`

- In der Regression steht jeder Suchlauf **zweimal** in `tuning.csv` — gesucht wurde aber nur einmal, auf der Rate. Ohne Entdopplung stünden zehn statt fünf Punkte je Parameter.
- `_faktorgruppen_balken` hält zusammen, dass der Mengenbalken **Koeffizienten** und die Strukturbalken **SHAP-Werte** zeigen — daher die Fußzeile in A6.

Was man über diesen Ordner wissen muss

1 · EIN LAUF, EINE SPEZIFIKATION

Nichts wird wiederverwendet, kein Zwischenstand eingelesen. `tuning.csv` ist ein **Ergebnis** dieses Laufs, kein Eingang. Wer die Spezifikation ändert, rechnet neu.

2 · GETESTET WIRD GEGEN STUFE 2

Nicht gegen die triviale Referenz. Der Primärtest läuft auf zehn Wiederholungsmitteln, gepaart, mit Effektgröße und Intervall daneben. Die 50 Einzelläufe laufen als gekennzeichnete Sensitivität mit.

3 · INTERPRETIERT WIRD NUR, WAS GEWINNT

SHAP nur für Modelle, die ihre Baseline schlagen — sonst erklärt man Rauschen. Im Mengenstrang bleibt dieser Block leer, und die Antwort auf UF1 kommt aus den Koeffizienten des Poisson-GLM.

DER SATZ, MIT DEM MAN DEN ORDNER VERTEIDIGT

„Der Vergleich misst die Verfahren, nicht die Modellierungsentscheidungen — deshalb sehen alle Modelle dieselben Merkmale, dieselben Folds und dieselbe Expositionsbehandlung.“ Wo das einmal nicht galt, ist es als Fehler in der Spezifikation korrigiert und dokumentiert worden, nicht als Ergebnis über die Verfahren berichtet.